COMPSCI 308 Design and Analysis of Algorithms

Quiz 01

Xing Shi Cai

2026-01-19

👀 Don't Cheat

During the quiz/exam do *not*

- 🩹 give or receive aid;
- ⏱️ read the questions before the quiz begins;
- 🛑 keep writing when the quiz ends;
- 📱 use a phone, calculator, computer or the internet;
- 🚽 hide textbooks or other aids in the toilet.

⚡ Follow Rules

- 👨‍🎓 Enter your name and **SID** (not your Net ID) in the designated space below.
- 🆎 Support your proofs with both equations and descriptive reasoning.
- ✏️ Use only pen for writing as pencil marks are not clearly visible.
- ❓ Restrict your inquiries to clarifications regarding typos.

Name: _____ SID: _____

Figure 1: Good Luck

1 Best vs Approximate Solutions (1 pt)

This adaptation is adapted from Exercise 1.1-5 of the fourth edition of *Introduction to Algorithms*.

💡 Hint: There are many correct answers. Think about situations where exactness is critical versus those where efficiency matters more than perfection.

1.1 (0.5 pt)

Suggest a real-world problem in which only the best solution will suffice.

Answer:

Sorting a list of financial transactions by date. An unsorted or partially sorted list can lead to incorrect balance calculations.

1.2 (0.5 pt)

Then, propose one in which an approximately best solution is good enough.

Answer:

Finding the shortest route for a delivery truck to visit a set of locations (Traveling Salesperson Problem). An approximate solution that is close to optimal is often sufficient and much faster to compute.

2 Radix Sort Example (1 pt)

👀 Radix Sort Algorithm

```
procedure RADIX-SORT(A, d)    ▷ A is an array A[1, ..., n], d is the number of digits
    for i = 1 to d do
        Initialize B[0, ..., 9] as empty buckets
        for j = 1 to n do
            Extract the i-th digit x from A[j]
            Append A[j] to B[x]
        k ← 1
        for x = 0 to 9 do
            for each element y in B[x] do
                A[k] ← y
                k ← k + 1
```

Consider the following list of numbers:

$$[\ 231 \quad 798 \quad 602 \quad 768 \quad 818\ ]$$

Perform Radix Sort on this list. The numbers have $d = 3$ digits.

2.1 Round 1 (0.3 pt)

Sort the list by the least significant digit (ones place).

Answer:

$$\text{Result} = \boxed{[\ 231 \quad 602 \quad 798 \quad 768 \quad 818\ ]}$$

2.2 Round 2 (0.3 pt)

Sort the result from Round 1 by the next digit (tens place).

Answer:

$$\text{Result} = \boxed{[\ 602 \quad 818 \quad 231 \quad 768 \quad 798\ ]}$$

2.3 Round 3 (0.4 pt)

Sort the result from Round 2 by the most significant digit (hundreds place).

Answer:

$$\text{Result} = \boxed{[\ 231 \quad 602 \quad 768 \quad 798 \quad 818\ ]}$$

3 Asymptotic Notation (1pt)

This adaptation is adapted from Exercise 3.1-2 of the fourth edition of *Introduction to Algorithms*.

Selection Sort Algorithm

```
    SelectionSort(A)                          ▷ A is an array A[1, ..., n]
1:  for i = 1 to n − 1 do
2:      min ← i
3:      for j = i + 1 to n do
4:          if A[j] < A[min] then
5:              min ← j
6:      exchange A[i] with A[min]
```

Using reasoning similar to insertion sort, analyze selection sort's running time.

3.1 (0.4pt)

Let $c_1, \ldots, c_6$ be the constant cost of lines 1–6, respectively. Let t_i be the number of times line 5 executes for a fixed i. Write the total running time $T(n)$.

💡 There is no need to simplify the expression.

Answer:

$$\boxed{\begin{array}{l} T(n) = c_1 n + c_2(n-1) + c_3 \sum_{i=1}^{n-1}(n-i+1) + c_4 \sum_{i=1}^{n-1}(n-i) + c_5 \sum_{i=1}^{n-1} t_i + c_6(n-1) \\ \text{or } T(n) = c_1 n + c_2(n-1) + c_3 \frac{(n-1)(n+2)}{2} + c_4 \frac{n(n-1)}{2} + c_5 \sum_{i=1}^{n-1} t_i + c_6(n-1) \end{array}}$$

3.2 (0.3 pt)

Give the worst-case running time in Θ-notation.

Answer:

$$\boxed{\Theta(n^2)}$$

3.3 (0.3 pt)

Give the best-case running time in Θ-notation.

Answer:

$$\boxed{\Theta(n^2)}$$

4 Approximate tic Notations (1 pt)

This adaptation is adapted from Exercise 3.2-6 of the fourth edition of *Introduction to Algorithms*.

Prove that $o(g(n)) \cap \omega(g(n))$ is the empty set.

Answer:

Suppose $f(n) \in o(g(n)) \cap \omega(g(n))$. Then for every $c > 0$ there exist n_0, n_1 such that for all $n \geq \max(n_0, n_1)$,

$$0 \leq f(n) < cg(n) \quad \text{and} \quad 0 \leq cg(n) < f(n).$$

Taking $c = 1$ gives $f(n) < g(n)$ and $g(n) < f(n)$ for all sufficiently large n, a contradiction. Therefore no such f exists, and $o(g(n)) \cap \omega(g(n)) = \emptyset$.

5 Merge Sort Pseudocode (1 pt)

The pseudocode below has five missing lines. Fill in the blanks.

Merge Algorithm (with blanks)

```
Merge(A, p, q, r)                                  ▷ A is an array A[1, ..., n]
n_L ← q − p + 1                                    ▷ length of A[p : q]
n_R ← r − q                                        ▷ length of A[q + 1 : r]
let L[0 : n_L − 1] and R[0 : n_R − 1] be new arrays
for i = 0 to n_L − 1 do
    (1) _____________________
for j = 0 to n_R − 1 do
    (2) _____________________
i ← 0                          ▷ i indexes the smallest remaining element in L
j ← 0                          ▷ j indexes the smallest remaining element in R
k ← p                          ▷ k indexes the location in A to fill
while i < n_L and j < n_R do
    if (3) _____________________ then
        (4) _____________________
        i ← i + 1
    else
        (5) _____________________
        j ← j + 1
    k ← k + 1
while i < n_L do
    A[k] ← L[i]
    i ← i + 1
    k ← k + 1
while j < n_R do
    A[k] ← R[j]
    j ← j + 1
    k ← k + 1
```

5.1 (0.2 pt)

Fill in blank (1).

Answer:

$$(1) = \boxed{L[i] \leftarrow A[p+i]}$$

5.2 (0.2 pt)

Fill in blank (2).

Answer:

$$(2) = \boxed{R[j] \leftarrow A[q+j+1]}$$

5.3 (0.2 pt)

Fill in blank (3).

Answer:

$$(3) = \boxed{L[i] \leq R[j]}$$

5.4 (0.2 pt)

Fill in blank (4).

Answer:

$$(4) = \boxed{A[k] \leftarrow L[i]}$$

5.5 (0.2 pt)

Fill in blank (5).

Answer:

$$(5) = \boxed{A[k] \leftarrow R[j]}$$

6 Bottom-Up Cut Rod Variations (1 pt)

Below is the original algorithm.

Bottom-Up-Cut-Rod

```
Bottom-Up-Cut-Rod(p, n)
let r[0..n] be a new array
r[0] ← 0
for j = 1 to n do
    q ← −∞
    for i = 1 to j do
        q ← max(q, p[i] + r[j − i])
    r[j] ← q
return r[n]
```

6.1 (0.2 pt)

Each piece must have a minimum length l. Fill in the missing line.

Minimum Length l

```
Bottom-Up-Cut-Rod-Min(p, n, l)
let r[0..n] be a new array
r[0] ← 0
for j = 1 to l − 1 do
    (1) ____________________
for j = l to n do
    q ← −∞
    for i = l to j do
        q ← max(q, p[i] + r[j − i])
    r[j] ← q
return r[n]
```

Answer:

$$(1) = \boxed{r[j] \leftarrow -\infty}$$

6.2 (0.2 pt)

Each cut has a cost c (charged when a cut is made, i.e., $i < j$). Fill in the missing line.

Cut Cost c

```
Bottom-Up-Cut-Rod-Cost(p, n, c)
let r[0..n] be a new array
r[0] ← 0
for j = 1 to n do
    q ← −∞
    for i = 1 to j do
        if i = j then
            q ← max(q, p[i])
        else
            (2) ____________________
    r[j] ← q
return r[n]
```

Answer:

$$(2) = \boxed{q \leftarrow \max(q, p[i] + r[j-i] - c)}$$

6.3 (0.2 pt)

The price of a piece depends on the total length of the rod, so $p[i]$ is replaced by $p[n, i]$. Fill in the missing line.

Price Depends on Total Length

```
Bottom-Up-Cut-Rod-Price(p, n)
let r[0..n] be a new array
r[0] ← 0
for j = 1 to n do
    q ← −∞
    for i = 1 to j do
        (3) _____________________
    r[j] ← q
return r[n]
```

Answer:

$$(3) = \boxed{q \leftarrow \max(q, p[j,i] + r[j-i])}$$

Note that there is a bit ambiguity of what depending on total length means. So all answers are considered correct for this problem.

6.4 (0.4 pt)

The number of pieces is limited to at most m. Write a complete modified algorithm (you may introduce a 2D table).

Answer:

```
Bottom-Up-Cut-Rod-Pieces(p, n, m)
let r[0..n, 0..m] be a new array
for k = 0 to m do
    r[0, k] ← 0
for j = 1 to n do
    r[j, 0] ← −∞
for k = 1 to m do
    for j = 1 to n do
        q ← −∞
        for i = 1 to j do
            q ← max(q, p[i] + r[j − i, k − 1])
        r[j, k] ← q
return r[n, m]
```

7 Substitution Method Recurrences (1 pt)

Use the substitution method to show that each recurrence has the stated asymptotic solution.

For this problem, there is no need to consider the base case. Only do the induction step, and be careful with your induction hypothesis.

7.1 (0.5 pt)

$$T(n) = 4T(n/2) + \Theta(n) \quad \text{has solution} \quad T(n) = \Theta(n^2).$$

Answer:

Let the additive term be $g(n)$, so $T(n) = 4T(n/2) + g(n)$. Since $g(n) = \Theta(n)$, there exist constants $a, b > 0$ and n_0 such that for all $n \geq n_0$,

$$an \leq g(n) \leq bn.$$

Upper bound: assume the induction hypothesis $T(n/2) \leq c\left(\frac{n}{2}\right)^2 - k\frac{n}{2}$. Then

$$T(n) \leq 4\left(c\left(\frac{n}{2}\right)^2 - k\frac{n}{2}\right) + bn = cn^2 - (2k - b)n \leq cn^2 - kn,$$

for $k \geq b$, so $T(n) = O(n^2)$.
Lower bound: assume the induction hypothesis $T(n/2) \geq c'\left(\frac{n}{2}\right)^2$. Then

$$T(n) \geq 4c'\left(\frac{n}{2}\right)^2 + an = c'n^2 + an \geq c'n^2,$$

so $T(n) = \Omega(n^2)$. Hence $T(n) = \Theta(n^2)$.

7.2 (0.5 pt)

$$T(n) = T(n/2) + \Theta(1) \quad \text{has solution} \quad T(n) = O(\lg n).$$

Answer:

Let the additive term be $g(n)$, so $T(n) = T(n/2) + g(n)$. Since $g(n) = \Theta(1)$, there exists a constant $b > 0$ and n_0 such that for all $n \geq n_0$,

$$g(n) \leq b.$$

Assume the induction hypothesis $T(n/2) \leq C\lg(n/2)$. Then

$$T(n) \leq C\lg(n/2) + b = C(\lg n - 1) + b = C\lg n + (b - C) \leq C\lg n$$

for $C \geq b$. Hence $T(n) = O(\lg n)$.

8 Change Making (1 pt)

You want to purchase an item priced at \$89. The vendor accepts coins in denominations of \$2, \$5, \$11, \$23, and \$50.

8.1 (0.5 pt)

Write pseudocode for an algorithm that computes the fewest number of coins needed to make exactly the target amount. Use the following as a template.

👀 Minimum Coins Pseudocode

```
Min-Coins(D, A)
let m[0..A] be a new array
m[0] ← 0
...
return m[A]
```

Answer:

```
Min-Coins(D, A)
let m[0..A] be a new array
m[0] ← 0
for a = 1 to A do
    m[a] ← +∞
    for each d ∈ D do
        if d ≤ a then
            m[a] ← min(m[a], m[a − d] + 1)
return m[A]
```

8.2 (0.5 pt)

Compute the fewest number of coins needed for the given amount.

Answer:

$$\boxed{4}$$

🌻🌷✨🌻🌷🐰🐰❤️⭐⭐ **The End of The Quiz** 🌻🌷✨🌻🌷🐰🐰❤️⭐⭐